

Automatic Creation of Indices in PostgreSQL

Team Members:

Lohit Adhitya (23b0952)
GnanaKoushik (23b1000)
AkshayKarthik (23b0981)
Maneendhar (23b0969)

1 Project Details

1.1 Objective

In many database applications, performance can degrade significantly because developers or administrators forget to create an index needed for frequent transactions. Adding the missing index often magically improves performance.

Cloud databases like SQL Server now handle this automatically. In Microsoft Azure SQL Database, a decentralized control plane continuously monitors query execution statistics to recommend, build, and validate indices asynchronously without disrupting the user's workload [1]. Inspired by such systems, the goal of this project is to modify the access layer of PostgreSQL to autonomously keep track of relation scans that could have benefited from an index. If there are many such scans for a particular relation and predicate, the system will automatically create the required index.

1.2 Scope of Work and Thresholds

To determine when an index should be created, we will use a cost-based heuristic. We will track the number of full relation scans that have an equality predicate on a relation. The trigger for creating the index will be evaluated using the following inequality:

$$B_{\text{idx}} \times N_{\text{scans}} > C_{\text{create}} \quad (1)$$

where:

- B_{idx} represents the estimated performance benefit (cost reduction) provided by the index per query.
- N_{scans} is the accumulated number of queries performing a sequential scan on the target relation.
- C_{create} is the estimated computational cost of creating the index.

Alternatively, this can be expressed simply as ensuring the aggregated cost of the sequential scans exceeds the creation cost:

$$C_{\text{total_scans}} > C_{\text{create}} \quad (2)$$

Initially, we will use synthetic constant values for these costs to build and test the core logic before integrating dynamic cost estimations.

2 High-Level Implementation Ideas

To accomplish this, we will divide the implementation into the following core areas:

- **Modifying the Node Scan Code:** We will modify the sequential scan execution code in PostgreSQL to identify and track full relation scans that utilize an equality predicate. We will maintain a counter for these specific relation-attribute pairs.
- **Cost Evaluation:** We will implement the threshold logic to continuously check if the accumulated cost of the sequential scans has crossed the threshold compared to the configured baseline cost of creating the index.
- **Asynchronous Index Creation:** Because creating an index is an expensive operation, it cannot be done by the process currently performing the scan. Similar to the Azure SQL architecture, when the threshold is satisfied, the system will hand off the actual index creation to a separate, background process so that the current transaction is not blocked.

References

- [1] Das, S., Grbic, M., Ilic, I., Jovandic, I., Jovanovic, A., Narasayya, V. R., Radulovic, M., Stikic, M., Xu, G., & Chaudhuri, S. (2019). *Automatically Indexing Millions of Databases in Microsoft Azure SQL Database*. In Proceedings of the 2019 International Conference on Management of Data (SIGMOD).